Actor IPC between implementation instances

Status: S1 and S2 implemented (D154, D155); S3 wire harness partial (D156); S2.5 language remote send and reply delivery landed (D157, D158); S4 planned. This document records the model, the boundary, the staging, and the open decisions.

The decision

The unified Julia/Elixir language (D93) is dropped. Scale does not host Julia and Elixir behind one source language, one AST, one type hierarchy, one dispatch model, or one module and macro system, and it does not provide Julia or Elixir front ends. The proposal in `docs/rejected-ideas/unified-language.typ` is kept as the record of the rejected direction.

Scale's own core language remains: the concatenative token surface and the general IR, with the Prose sentence grammar and the prose surface as front ends over it. The term "unified language" is retired; the direct surface is the **core language**.

Different implementations are separate **implementation instances**, each running in its own host language. They integrate through the actor model's IPC: an actor handle may address an actor in another instance, and a message crosses the instance boundary through the host adapter's transport. The local actor model, the image promotion rule, and the `scale_transport_send` ABI seam are the starting point.

The model

An **implementation instance** (an instance) is one running actor system in a host language. It owns its source language, parser, evaluator, image, and host adapter. The unit of integration is an actor, not a source construct.

1. **Address.** An actor handle is the only address. A local handle names an actor in the current instance; a remote handle names an actor in another instance. A remote handle is a capability, not a name that any instance may forge.
2. **Envelope.** A message carries the target handle, the sender handle, an optional reply channel, a payload, a message identity, and an operation (send, reply, monitor, monitor acknowledgement, or down). The receiving instance delivers a send to the target actor's mailbox. The transport may add framing, authentication, and routing metadata; it does not change the actor semantics.
3. **Portability.** A payload is a portable value: a durable image entry, a canonical document, or a scalar. A host object, a closure, a frame, a stack, a tensor view, or an execution-state root never crosses an instance boundary.
4. **Failure.** A failure is a named value. An unknown instance, an unreachable instance, a transport fault, and a remote exit are distinct. A partition is an outcome, not a silent message loss.
5. **Authority.** The transport authenticates instances and the capability handshake grants authority. The actor model does not create authority from an instance name or an integer.

The kernel frames and checks the handle; the host adapter owns transport semantics. `scale_transport_send` is the current ABI seam (D109): the host receives a framed target and payload. The wire encoding is the **Scale actor wire protocol**, implemented in-repo and Cap'n Proto-compatible for the frozen subset (D152, D153); compatibility is gated by a reference conformance test. The message keeps a logical envelope/body split in one frame; only bulk data may be out-of-line. Schema tooling may borrow Protobuf's plugin contract and field-number metadata, but those stay off the wire. JSON remains a debug fallback and CBOR is not used. The IPC protocol is a host-adapter contract, not a second evaluator and not a second image.

What is not built

1. No Julia or Elixir front end.
2. No shared type hierarchy, dispatch model, module system, or macro system across host languages.
3. No claim of total coverage of either ecosystem.
4. No transparent remote object; a remote handle is an actor capability, not a proxy for a host value.
5. No distributed garbage collector; promotion and explicit lifetimes remain the rule.
6. No change to the local actor model in specification chapter 11.

Staging

1. **S0 — record the direction.** D151–D153 and this document. No code change.
2. **S1 — two-instance loopback (landed, D154).** Two processes on one host exchange a numeric message and a reply through a host adapter over a Unix socket, using the Scale actor wire protocol. The listener delivers to the root actor's mailbox, the actor adds one, and the reply is 42. The test also pins a truncated frame and a non-struct root pointer. Sender identity and a named failure for an unknown target remain open.
3. **S2 — remote supervision (landed, D155).** The listener creates a worker actor, assigns it capability 1, and monitors it from the root. The client monitors capability 1, sends 21, receives 42, and then receives `down normal` when the worker terminates. An unreachable instance is a named `Unreachable` outcome. A partition is still distinct from a remote exit.
4. **S2.5 — language remote send (landed, D157, D158).** `send-to` recognizes a host-registered remote handle and records a `TransportRequest`; the host drains it, delivers the numeric reply to the sender's mailbox, and resumes the sender. `send-to; receive-message` returns the remote result. Text replies and down delivery into the language remain.
5. **S3 — host adapters (partial).** The Elixir wire harness is landed (D156): `adapters/elixir` has a codec, a peer, and a BEAM listener, and both cross-language directions pass. The full BEAM language port and the Rustler C ABI adapter are deferred until the Prelude is in Scale and a native kernel can be tested. Julia is not started.
6. **S4 — negotiation and recovery.** Version and capability negotiation, reconnect, and a bounded mailbox with a named refusal.

S1, S2, and the language-level remote send are implemented; the Elixir wire harness covers S3. The full BEAM port is deferred; S4 remains. No earlier stage changes the core language, the image, or the suite.

Open decisions

1. **Wire encoding.** The Scale actor wire protocol (D152, D153), implemented in-repo and Cap'n Proto-compatible for the frozen subset, conformance-gated. Protobuf-style schema tooling stays off the wire. JSON is a debug fallback; CBOR is not used. `docs/binary-format.typ` records the subset, the staging, and the conformance plan.
2. **Instance identity and discovery.** A host-provided name, a registry actor, or an external service.
3. **Delivery and ordering.** At-most-once by default, or an acknowledged mode; whether ordering is per sender-receiver pair or per instance.
4. **Payload portability.** Which image entries are sendable, and the canonical form for the rest.
5. **Failure and partition semantics.** Timeout, retry, and the exit-reason taxonomy.
6. **Security.** Instance authentication, the capability handshake, and the refusal when a capability is absent.
7. **Backpressure.** Mailbox bounds and the refusal when a remote mailbox is full.

Relationship to other documents

`docs/specification.typ` chapter 11 is the local actor model and section 11.7 is the instance boundary. `docs/binary-format.typ` is the wire protocol plan. D151 in `docs/decision-records.typ` revokes D93; D152 records the wire decision, D153 the schema-tooling refinement, D154 the first slice, D155 the worker capability and remote supervision, D156 the Elixir wire harness, D157 the language-level remote send, D158 the reply delivery, and D159 the wire golden vectors and reference interop. The dropped proposal is in `docs/rejected-ideas/unified-language.typ`. The BEAM matrix in `docs/rejected-ideas/beam-compatibility.typ` describes the local subset that an Elixir instance would expose; it is not a claim that Scale is BEAM. The release and ABI rules are unchanged: one version, one kernel, one Prelude, and one adapter contract.